

一个接口约定，两种实现 `sim/base.py` 中的 Protocol；两种后端共用同一套控制循环、运行器和记录器

	VehicleBackend 单机；SI 单位；世界系 Z 轴向上；状态取整机质心；力/力矩绕质心 <code>reset(initial_position_w=None)</code> 放置机体，复位电机与采样的 k_f, τ <code>read_state(t) → VehicleState</code> 质心位置、<code>wxyz</code>（机体 → 世界）、v_w, Ω_b <code>mass_properties() → (m, J, c)</code> 整机质量、绕质心的完整惯量、机体系中的质心 <code>allocation_matrix_b</code> 6×4：单架推力 → 绕质心的机体力/力矩 <code>motor_parameters()</code> <code>thrust_to_motor_speeds(f)</code> <code>apply_motor_speeds(n, w_ext)</code> <code>telemetry()</code> 采样的 $k_{f,l}, [f, \bar{f}], [n_{min}, n_{max}]$、当前转速 $n_l = \sqrt{f_l/k_{f,l}}$，限制在运行上下限内；0 保持为 0 一次电机积分 + 提交作用力；每个物理步恰好一次 转速、指令与实际推力、电机合力/力矩、数据来源
	SimulationDriver <code>step()</code> 恰好推进一个 Δt <code>is_running()</code> <code>is_paused()</code> <code>idle()</code> 暂停时保持界面刷新 可选：<code>ground_start_position</code>、<code>assert_control_compatible</code>、<code>describe</code>
	实现
	SyntheticBackend + SyntheticDriver <code>sim/synthetic.py</code> 仅需 CPU 与 NumPy · 本地与 CI · 20 s 的 helix 约 12 s 跑完 ARLBackend + IsaacDriver <code>sim/isaac/lab/</code> Isaac Sim 6.1 + Isaac Lab 3.0 + PhysX · GPU 服务器 uc1
模型来源	SyntheticPlantParameters：刻意不对称的测试数值，不是 ARL 数据。$m = 1.24 \text{ kg}$；含惯性积的完整 J；质心偏移 $c = (6, -4, 2) \text{ mm}$；$k_f = (1.1, 1.3, 1.6, 1.8) \times 10^{-5} \text{ N/rps}^2$；$\tau_{inc} = 50\text{--}80 \text{ ms}$，$\tau_{dec} = 4.5\text{--}6 \text{ ms}$；$f \in [0.1, 10] \text{ N}$；$k_\tau = 0.07 \text{ m}$。 ARL-Robot-1 USD：base_link + 4 个旋翼刚体，以固定关节连接。PhysX 解算出的各刚体质量、质心和主惯量，经旋转与平行轴定理合成为整机的 m、c、J；旋翼原点取自连杆位姿；k_f, τ 在复位时从配置的范围內采样。
分配矩阵	$A_i = [a_i; (r_i - c) \times a_i + \sigma_i k_\tau a_i]$，由合成对象的旋翼位置（$\pm 0.1 \text{ m}$）与 c 计算；$\sigma = (-1, +1, +1, -1)$。 同一公式，改用实际连杆几何与 PhysX 质心计算；上游参考矩阵（$\pm 0.13 \text{ m}$）只作为差异记录在日志中。
电机模型	TranscribedThruster：上游 Thruster 积分函数的 NumPy 转写（有 Isaac Lab 源码时由契约测试逐项比对），经共用的 NativeRpsActuator 驱动。 上游 isaac/lab_contrib 的 Thruster，由同一个 NativeRpsActuator 包装：直接存储转速，按升速/降速选 τ_{inc}/τ_{dec}，混合系数 $1/(\Delta t + \tau)$，变化率限幅，RK4，$f = k_f n^2$。
施力方式	提交的合力/力矩 $w = A f_{\text{actual}} + w_{\text{ext}}$ 作用在单一刚体的质心上（机体系表示）。 $f_i a_i$ 与反扭矩 $\sigma_i k_\tau f_i a_i$ 作用在各旋翼刚体原点，每步用一次 PhysX 调用提交全部刚体： <code>apply_forces_and_torques_at_position</code> <code>base_link</code> 只受 w_{ext}；控制器的合力/力矩从不直接施加到刚体。 只有 w_{ext}
刚体推进	对 (p, v, q, Ω) 做 RK4：$\dot{p} = v + R F_b / m$，$\dot{q} = \frac{1}{2} q \otimes (0, \Omega)$，$\dot{\Omega} = J^{-1}(M - \Omega \times J \Omega)$。地面：只在质心施加竖直非弹性冲量（无摩擦、接触力矩和阻力）。 PhysX 关节体：重力、固定关节、碰撞体与地面接触、原生阻尼（ARL 配置中为 0）。 <code>IsaacDriver.step()</code> = <code>sim.step(render=False)</code> + <code>robot.update(dt)</code>。
状态读取	状态直接以质心量存储：位置、v_w、<code>wxyz</code> 四元数、Ω_b。 根连杆位姿 (xyzw) 在接口处转换：$p = p_{\text{root}} + R c$，$v = v_{\text{root}} + \omega \times R c$，$q: \text{xyzw} \rightarrow \text{wxyz}$，$\Omega_b = R^T \omega_w$。
驱动器与应用	SyntheticDriver：始终运行、从不暂停。<code>run_simulation()</code> 组装控制循环、运行器和指标/画图钩子。 app.py：AppLauncher (Kit)、搭建场景、WebRTC 界面刷新、<code>--wait-for-start</code>，暂停 $\Leftrightarrow$ <code>not sim.is_playing()</code>；<code>idle()</code> 只做渲染。安全检查：非固定基座、动态刚体、A 秩为 4、每步只提交一次。
本后端独有	闭环黄金回归测试 (<code>tests/data/golden_synthetic.npz</code>)；<code>sweep</code> 多进程并行扫参。 多机位相机 + 同步 MP4；轨迹叠加几何；PhysX 解算器加速度；USD 各层 SHA-256；<code>inspect</code>。